

A Cost-Benefit Study of Template Metaprogramming

1st Miguel Vezanzones Parellada
Technical University of Munich
Munich, Germany
miguel.vezanzones@tum.de

Abstract—

Index Terms—component, formatting, style, styling, insert.

I. INTRODUCTION

explore how increasing compiler visibility through templates and template metaprogramming techniques affects the performance and generated code quality of numerical kernels.

will compare implementations of the same algorithm with various degrees of runtime/compile time information, focusing on runtime, compile time, code size, and constant propagation in the generated assembly.

The study will compare runtime-generic implementations against progressively more compile-time-specialized variants.

C++ philosophy: The aim is to allow a programmer to work at the highest feasible level of abstraction by providing - A simple and direct mapping to hardware - Zero-overhead abstraction mechanisms

Various forms of user-specified compile-time computation are essential in critical embedded systems applications, much low-level code, and many high-end numerical applications. [1, p. 6]

II. COMPILE-TIME INFORMATION

Template metaprogramming allows implementing thin layers of abstraction whose complexity is resolved at compile time.

The extent to which this can occur is limited by compiler visibility and available information.

Compiler visibility is heavily influenced by dynamic binding. Compile time information must be hard-coded into the source, and it must also be propagated at compile time to be usable in `constexpr` contexts. Compile time information can be lowered to runtime information in a nested context, reducing the compiler's visibility in said context. This process is irreversible, and defines a flow of information from its source to its destination, where it is eventually consumed. External inputs are never present in the source and are therefore never usable at compile time. Only information present in the source, and computable results can be used at compile time.

Template metaprogramming allows manipulating values and types at compile time. Values in `constexpr` contexts additionally. Types are very powerful in a statically, strongly typed language

This defines a unique direction of the flow of information from its source to its sink, where it is eventually consumed.

If a value is ever lowered to the runtime, it cannot be elevated to the runtime at a later stage as the information is lost from the present context and all subsequent contexts.

Template metaprogramming is a powerful modern C++ tool, which is leveraged to afford efficient abstractions in highly-optimized applications [2].

The effects of compiler visibility and template metaprogramming are often understood in terms of zero-cost abstractions, but the implication goes beyond that, and are described below.

Simple and direct mapping to hardware and zero-cost abstractions [1, p. 2]

Compute a type at compile time [1, p. 6]

A. Zero-Cost Abstractions

C++ follows the Zero-Overhead Principle. The zero-overhead principle is a C++ design principle that states:

You don't pay for what you don't use. What you do use is just as efficient as what you could reasonably write by hand.

In general, this means that no feature should be added to C++ that would impose any overhead, whether in time or space, greater than a programmer would introduce without using the feature.

The only two features in the language that do not follow the zero-overhead principle are runtime type identification and exceptions, and are why most compilers include a switch to turn them off. Zero-cost abstractions is a core design philosophy of the C++ language

B. Compile Time Work

4 Compile-time Computation Sometimes, we prefer a computation be done at compile-time. The reasons vary, for example: • Efficiency: To pre-calculate a value (often a size). For simple cases, that is done by an optimizer. Examples include object and array sizes and table values. • Type-safety: To compute a type at compile time. • Simplify concurrency: you can't have a race condition on a constant.

[1, p. 6]

Templates as Type function: template functions generate functions, template classes generate classes [1, p. 21]

1) *Move computation from runtime to compile time:* Moving any computation from runtime to compile time should increase compile times in favor of runtime. Whether to do it comes down to whether it is possible and what cost is more important. Compile time execution is slower as it executes on an interpreter inside the compiler.

2) *Avoid redundant computation at runtime:* (some of the computation may be done repeatedly) API designers, for ease of usability may desire to repeat computation to simplify the callsite. In our case later on, an efficient implementation may have: $f(\text{in } A, \text{in } B, \text{out } C)$ with a pre-constructed C instead of $C = f(\text{in } A, \text{in } B)$. As calculating the structure of C would need to be done per call, rather than per caller.

3) *Compile-Time Complexity:* Cost of recompilation/code size vs runtime

reduced over time from explicit `sizeof` to `if constexpr` template for

C. Code Generation

1) *Enable optimizations:* - automatic unrolling - loop stamping

2) *Code Size:* - compile time logic to compute values is not preserved in the executable. Reduce object sizes - reduced indirections and stuff. Hard coding a result may be cheaper than hard coding the procedure to obtain it. Increase code size. Template deduplication. Inlining/unrolling.

3) *Type simplification/stronger properties/weaker properties/requirements:* - Trivial types - Standard layout - Trivial copyability

4) *Weaker move:* Pointer/reference semantics vs value semantics

5) *constexpr enabling and propagation:* Initialization

6) *Compile time contracts:*

D. Architectural

1) *Reduce memory operations:* Immediate arguments are inevitably fetched with their instruction, bypassing loads from the data pipeline to be fetched. This reduces the overall number of load operations in the assembly. What type categories and bit widths are allowed is architecture-dependent. `intel64` accepts small integral values and no floating point, while `amd64` also supports floating point immediates to some extent. I believe

2) *Reduce function arguments:* Static global data region has predictable locations. Heap is clearly in a disadvantage. Stack arguments must be input arguments as their location is unknown to an arbitrary caller. Static/global symbols in `.rdata` or `.data` have predictable locations, which can potentially be hardcoded in the load instruction.

3) *Reduce register pressure:* Immediate arguments reduce register pressure as they never become part of the in-flight or architectural state, but are rather consumed and discarded.

4) *Read-only data and coherence protocols:* Read only data regions are simpler to manage by a multithreaded architecture as it promises stronger invariants. Race conditions, initialization

E. Memory Regions

1) *Reduce the need for allocation:* The stack is compiler controlled. It is restricted to objects of known size and local-context-determined lifetime. The heap softens these restrictions in exchange for Operating System (OS) interaction, increased compiler uncertainty and indirections. Lowering values from runtime to compile time may enable the use of more restrictive sets of memory.

Heap: Free Stack: Known structure, dynamically evaluated. Static data: Statically known layout (v2 sizes). `constexpr` data: Statically known values (v3 sizes).

Only if we explicitly allocate an (unnamed) Point on the free store (the heap), as done for the Point pointed to by `p`, do we incur memory overhead (and allocation overhead). Such very simple user-defined types are critical to type-rich programming and very common [1, p. 3].

2) *Static/Global Data:* Increase data size. Static `constexpr`

3) *Increased stack pressure:* possible overflow

4) *Reduce indirections from heap to the stack:* Increase compile time complexity. If scaling is bad, the compile time interpreter can exhaust resources.

III. DESIGN SPACE

runtime vs compile-time parameters flexibility vs performance abstraction vs generated code simplicity

IV. COMPUTE KERNEL

The extent to which compile time information can be exploited effectively is highly application-dependent. In particular, a reference application must be affected in a meaningful way by the absence or presence of this compile time information. Tensor contraction is the algorithm chosen as reference application as there are two clear progression steps that can be exploited both in both the container and algorithm implementations.

[2]

The microkernels

Compile-time information is and the reference algorithm must be able to effectively exploit it.

numerical kernels as interesting applications

Types - would be pathological to consider it. Rank Sizes

Tensor contraction

qualitative and quantitative changes

dynamic to compile time polymorphism

memory allocation immediate variables

v1: not knowing the rank effectively requires a second loop during iteration, effectively a multi-index

V. IMPLEMENTATION

We consider three levels of compile time information that the implementation can exploit in a meaningful way. Each level will build on top of the previous one, always elevating information from runtime to compile time. These levels are denominated *runtime rank*, *compile-time size* and *compile-time sizes* (v1, v2, v3 respectively), according to how the available information affects each implementation.

These can be thought of as different specification standards with different requirements for compliance.

A. Runtime Rank (v1)

This level specifies the lowest degree of compile-time information expected by each of the implementations. It requires static typing and static binding. This is a sensible baseline as it is often taken for granted.

Static typing implies that the concrete types being used are known to the compiler (*i.e.*, no `std::any` or `void*`). Static binding implies that function calls are resolved at compile time (*i.e.*, no virtual dispatch or function pointers).

In this context, the compiler has full visibility of the concrete types and call structure, enabling optimization across function boundaries and efficient code generation. However, no algorithm-specific information such as ranks, dimensions or contraction patterns will be accessible during code generation.

1) `v1::tensor(t1)`: This variant must use the heap to store both its data and its metadata. The data buffer will be a contiguous buffer of `T`. Similarly, the metadata buffer will contain the tensor sizes and strides for efficient access with a multi-index. Accessing these buffers through a pointer indirection does not meaningfully affect runtime on its own, as compared to the heap. However, we will have 7 different allocations¹ which will demand more memory subsystem resources.

2) `v1::contraction(c1)`: Since the rank of the tensors is unknown at compile time, the first-level contraction implementation uses a multi-index to iterate over the tensors. The multi-index uses an internal `for` loop for each increment, introducing significant overhead in iteration.

B. Compile-Time Rank (v2)

The second level requires that the rank of the tensors and the contraction degree be known at compile time.

1) `v2::tensor(t2)`: Knowing the rank of the tensor allows the strides and sizes of the tensors involved to be stored within the tensor object itself. Thanks to this compile time information, the metadata of the tensor will be stored in the stack, rather than the heap, and accessible without a pointer indirection.² The data itself must still be heap-allocated, however, as the size of each dimension is not known.

2) `v2::contraction(c2)`: Knowing the rank of the tensors significantly simplifies iterating over the tensors³. This implementation sets up two sets of nested `for` loops using template metaprogramming, removing the need to iterate using a multi-index.

C. Compile-Time Sizes (v3)

The third level requires that the sizes of the tensors and the indices to contract over, be known at compile time.

¹Two allocations for each tensor and one for the contraction index set. Each tensor uses two buffers in the heap, these could be merged into one, but the additional complexity was deemed unimportant.

²Moving data from the stack to the heap also removes a costly memory allocation, but these are not included in the benchmark loop.

³The rank of a tensor corresponds to the number of nested `for` loops required to iterate over it in a structured way.

1) `v3::tensor(t3)`: This affects the tensor implementation in two major ways. For once, the data in the tensor may be stored in the stack, significantly improving locality, specially for small tensors. Additionally, tensor sizes and strides become part of the type, reducing the size of each instance.

2) `v3::contraction(c3)`: Knowing the size of the tensors allows propagating these values into the loop limits into the two sets of nested `for` loops. This enables inlining the loop limits into the assembly as integral immediate arguments, and enables aggressive loop unrolling and vectorization.

Additionally, iteration functions now require less input arguments, since the location of static data is known at compile time. This change makes feasible computing the target linear index into each tensor during iteration, removing an inner product computation between the strides and the multi-index on each tensor access.⁴

D. Signatures

The first level tensor has one template parameter for its value type

- `v1::tensor<T>`
- `v2::tensor<T, N>`
- `v3::tensor<T, A1, ..., AN>`
- `v1::contract<T>(in A, in B, out C, in cis)`
- `v2::contract<T, N>(in A, in B, out C, in cis)`
- `v3::contract<T, A1, ..., AN, B1, ..., BN, C1, ..., CN, CIS>(in A, in B, out C)`

E. Compatibility

Contraction and tensor implementations of different levels are compatible because each level extends the previous one with stricter requirements and compile time information can be lowered to runtime information. Since the opposite is not true, compatibility goes in one direction, making `v3::tensor` compatible with any contraction implementation, `v2::tensor` compatible with `v2::contraction` and `v1::contraction`, and `v1::tensor` only compatible with `v1::contraction`.

This will be used to assess what part of the performance improvements can be attributed to changes in the tensor implementation or the contraction implementation.

VI. EXPERIMENTAL SETUP

arch linux 6.18.35-1-lts GCC 16.1.1 c++26 mode i7 9750h 32kb L1 2MB L2 12 MB L3 unified

VII. RESULTS

Introduce metrics

⁴This same optimization was unsuccessful for the `v2::contraction` iteration scheme, presumably for the significant increase in function arguments.

A. Runtime

B. Compile Time

C. Code Generation

VIII. DISCUSSION

IX. CONCLUSION

REFERENCES

- [1] B. Stroustrup, *Foundations of C++*, Berlin Heidelberg, 2011. [Online]. Available: <https://www.stroustrup.com/ETAPS12-corrected.pdf>
- [2] A. Hendricks, “Lessons Learned: Template-Heavy C++ in Production HPC Runtime Systems,” en, in *Proceedings of the SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, St Louis MO USA: ACM, Nov. 2025, pp. 1009–1016, ISBN: 979-8-4007-1871-7. DOI: 10.1145/3731599.3767476 Accessed: May 6, 2026. [Online]. Available: <https://dl.acm.org/doi/10.1145/3731599.3767476>